Final Project: Network Door Security System (PYNQ #1)

Device Roles

PYNQ #1: Controls the alarm function of the door security system. Will emit a loud buzzing sound and flash a RGB board bright red when sound is detected with a sound sensor. This board will automatically be listening for the sound sensor board when the code is run.

PYNQ #2: Controls the sound sensor board. When a sound is detected it will send a signal to PYNQ board #2 and buzz the buzzer and flash the LED.

Wiring to this board:

Buzzer Module Wiring to PYNQ PMODA

- (-) pin connected to GND
- (+) pin connectted to PMOD PIN2
- Middle pin is not connected

RGB LED Connected to PYNQ PMODB

- (-) pin connected to GND
- R pin connected to Pin 3
- G pin connected to Pin 2
- B pin connected to Pin 1

In [1]:
```
from multiprocessing import Process
from multiprocessing import Event
import threading
import time
from datetime import datetime
from pynq.overlays.base import BaseOverlay
base = BaseOverlay("base.bit")
import socket
import sys
import os

btns = base.btns_gpio
stop_program = Event()
```

In [2]:
```
%%microblaze base.PMODA
// Buzzer Controls
#include "gpio.h"
#include "pyprintf.h"
#include <unistd.h>

static int inited = 0;
```

```
static gpio pins[8];

void init_pmoda()
{
    if(inited) return;

    for(int i = 0; i < 8; i++)
    {
        pins[i] = gpio_open(i);
        gpio_set_direction(pins[i], GPIO_OUT);
        gpio_write(pins[i], 0);
    }

    inited = 1;
}

void write_gpio(unsigned int pin, unsigned int val)
{
    if (!inited) init_pmoda();
    if (pin >= 8) return;
    gpio_write(pins[pin], val);
}

void reset_all()
{
    if (!inited) init_pmoda();
    for(int i = 0; i < 8; i++)
        gpio_write(pins[i], 0);
}

// Buzzer
void buzz(unsigned int pin,
          unsigned int freq_hz,
          unsigned int duration_ms)
{
    if (!inited) init_pmoda();

    if (pin >= 8) { pyprintf("pin must be 0-7\n"); return; }
    if (freq_hz == 0) { pyprintf("freq must be > 0\n"); return; }

    // 50% duty cycle
    unsigned int period_us = 1000000u / freq_hz;
    if (period_us == 0) period_us = 1;

    unsigned int half_period = period_us / 2;
    if (half_period == 0) half_period = 1;

    unsigned int cycles = (duration_ms * 1000u) / period_us;

    for (unsigned int i = 0; i < cycles; i++)
    {
        // write value of 1
        gpio_write(pins[pin], 1);

        //sleep for 1/(2*tone_freq)
        usleep((useconds_t)half_period);

        // write value of 0
        gpio_write(pins[pin], 0);
```

```
        //sleep for 1/(2*tone_freq)
        usleep((useconds_t)half_period);
    }

    gpio_write(pins[pin], 0);  // ensure off
}

void beep(unsigned int pin)
{
    // 2 kHz tone for 500 ms
    buzz(pin, 2000, 500);
}

void high_beep(unsigned int pin)
{
    buzz(pin, 3500, 300);
}
```

In [3]:

```
%%microblaze base.PMODB
// RGB LED Controls
#include "gpio.h"
#include "pyprintf.h"
#include <unistd.h>
static int inited = 0;
static gpio pins[8];
void init_pmoda()
{
    if(inited)
        return;
    for(int i = 0; i < 8; i++)
    {
        pins[i] = gpio_open(i);
        gpio_set_direction(pins[i], GPIO_OUT);
        gpio_write(pins[i], 0);
    }
    inited = 1;

}
//Function to turn on/off a selected pin of PMODA
void write_gpio(unsigned int pin, unsigned int val){
    if (val > 1){
        pyprintf("pin value must be 0 or 1");
    }
    if(!inited)
    {
        init_pmoda();
    }
    gpio_write(pins[pin], val);
}

//Function to read the value of a selected pin of PMODA
unsigned int read_gpio(unsigned int pin){
    gpio pin_in = gpio_open(pin);
    gpio_set_direction(pin_in, GPIO_IN);
    return gpio_read(pin_in);
}

// reset GPIO bins meaning writing 0 as output to all pins 0 - 7
void reset_pin(unsigned int pin)
```

```
{
    if(!inited)
    {
        init_pmoda();
    }
    write_gpio(pin, 0);
    //read_gpio(pin);
}

void run_pwm(unsigned int pin,
             unsigned int freq_hz,
             unsigned int duty_milli,
             unsigned int duration_ms)
{
    if (!inited) init_pmoda();

    if (pin >= 8) { pyprintf("pin must be 0-7\n"); return; }
    if (freq_hz == 0) { pyprintf("freq_hz must be > 0\n"); return; }
    if (duty_milli > 1000) duty_milli = 1000;

    // corner cases: 0% / 100%
    if (duty_milli == 0) {
        gpio_write(pins[pin], 0);
        usleep((useconds_t)duration_ms * 1000);
        return;
    }

    if (duty_milli >= 1000) {
        gpio_write(pins[pin], 1);
        usleep((useconds_t)duration_ms * 1000);
        // turn LED off
        gpio_write(pins[pin], 0);
        return;

    }

    // period in microseconds
    unsigned int period_us = 1000000u / freq_hz; // how many us per cycle
    if (period_us == 0) period_us = 1; // avoid 0 due to rounding ---- safety

    unsigned int on_us = (period_us * duty_milli) / 1000u;
    unsigned int off_us = period_us - on_us;

    pyprintf("on_us is %ui", on_us);
    pyprintf("on_us is %ui", off_us);

    // avoid 0 due to rounding ---- safety
    if (on_us == 0) on_us = 1;
    if (off_us == 0) off_us = 1;

    unsigned int cycles = (duration_ms * 1000u) / period_us;

    for (unsigned int i = 0; i < cycles + 1; i++) {
        gpio_write(pins[pin], 1);
        usleep((useconds_t)on_us);
        gpio_write(pins[pin], 0);
        usleep((useconds_t)off_us);
    }
}
```

```
void run_pwm_for6(unsigned int pin)
{
    run_pwm(pin, 100, 250, 1000);
    write_gpio(pin, 0);
    usleep((useconds_t)1000000u);
}
```

In [4]:
```
# Code that runs LED from HW1 to Test RGB LED
for pin_num in range(8):
    reset_pin(pin_num);
#write_gpio(1, 1)

# run_pwm(pin, frequency, duty cycle, duration)

# Turns on red LED and turns off after duration is over
run_pwm(3, 10, 988, 750)
```

In [5]:
```
# Test code that beeps the buzzer for 750 ms
buzz(2, 4500, 750)
```

In [6]:
```
# Test Code for alarm loop. Will buzz the buzzer and turn on the LED at the same time

while True:

    # Define thread of led and command to run LED
    # writes to pin 3 and turns on RED RGB LED for 750 ms at a frequency of 75Hz and 1
    thread_led = threading.Thread(
        target=run_pwm,
        args=(3, 75, 1000, 750)
    )

    # starts thread
    thread_led.start()

    # runs the buzzer at a frequency of 4.5 KHz for 750 ms
    buzz(2, 4500, 750)

    # stops thread
    thread_led.join()

    # Pauses thread for 1 second
    time.sleep(1)
```

```
---------------------------------------------------------------------------
KeyboardInterrupt                         Traceback (most recent call last)
Input In [6], in <cell line: 3>()
     19 thread_led.join()
     21 # Pauses thread for 1 second
---> 22 time.sleep(1)

KeyboardInterrupt:
```

In [7]:
```
def server():

    HOST = ''
    PORT = 50007

    # creating a socket
    sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

```
s_server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
print('Socket created')

# 1: Bind the socket to the pynq board <CLIENT-IP> at port <LISTENING-PORT>
s_server.bind((HOST, PORT))
print('Socket bind complete')

s_server.listen(1000)

# 2: Accept connections
print('Waiting for message from sound sensor board...')

conn, addr = s_server.accept()

with conn:
    print("\n-------------------------------------------------")
    print('Connected by', addr)

    while True:

    #Test Code
    # 3: Receive bytes from the connection
        #data = conn.recv(1024)
        # 4: Print the received message
        #print('Received', data)
        #if not data: break
        #conn.sendall(data)

        data = conn.recv(1024)

        # if not data:
        #    break

        if data == b'ARM':
            print("\n-------------------------------------------------")
            print("Button 1 Pressed - Security system armed")

        if data == b'DISCONNECT':
            print("\n-------------------------------------------------")
            print("Button 3 Pressed - Sound sensor board disconnected from server"
            break

        # print("Server Shutting Down")

        elif data == b'ALARM':
            print("\n-------------------------------------------------")
            print("UNAUTHORIZED ACCESS DETECTED")

            conn.setblocking(False)

            alarm_active = True

            # run alarm until button 2 is pressed
            while alarm_active:

                try:
                    msg = conn.recv(1024)

                    # Disarm or disconnect when the alarm board is actively buzzin
                    if msg == b'DISARM' or msg == b'DISCONNECT':
```

```
                            print("\n----------------------------------------------"
                            print("Button 2 Pressed - Security system disarmed")
                            alarm_active = False
                            break

                    except BlockingIOError:
                        pass

                    # Thread that runs the RGB LED board and buzzer indefinitely until
                    thread_alarm = threading.Thread(
                        target=run_pwm,
                        args=(3,75,1000,650)
                    )

                    thread_alarm.start()

                    buzz(2,4500,650)

                    thread_alarm.join()

                    # Pauses thread for 1 second
                    time.sleep(1)

                conn.setblocking(True)

            # Disarm when the alarm board is actively buzzing/flashing
            elif data == b'DISARM':
                print("Button 2 Pressed - Security system disarmed")
                alarm_active = False
                # Turn off LED and buzzer completely
                alarm_active = False

            # Disconnect when the alarm board is actively buzzing/flashing
            elif data == b'DISCONNECT':
                print("\n----------------------------------------------")
                print("Sound sensor board disconnected from server")
                break
```

In [11]:

```
# Server process turns on upon code execution
button_pressed = True

print("Starting Alarm server...")

# Server Process
p = Process(target=server)
p.start()
time.sleep(1)

# Stop server process
# p.terminate()
# print("Server process terminated")
# p.join()
# print("Cell execution complete")
```

```
Starting Alarm server...
Socket created
Socket bind complete
Waiting for message from sound sensor board...

------------------------------------------------
Connected by ('192.168.0.67', 35862)

------------------------------------------------
Button 1 Pressed - Security system armed

------------------------------------------------
UNAUTHORIZED ACCESS DETECTED

------------------------------------------------
Button 2 Pressed - Security system disarmed

------------------------------------------------
Button 1 Pressed - Security system armed

------------------------------------------------
UNAUTHORIZED ACCESS DETECTED

------------------------------------------------
Button 2 Pressed - Security system disarmed

------------------------------------------------
Button 1 Pressed - Security system armed

------------------------------------------------
UNAUTHORIZED ACCESS DETECTED

------------------------------------------------
Button 2 Pressed - Security system disarmed

------------------------------------------------
Button 1 Pressed - Security system armed

------------------------------------------------
UNAUTHORIZED ACCESS DETECTED

------------------------------------------------
Button 2 Pressed - Security system disarmed

------------------------------------------------
Button 1 Pressed - Security system armed

------------------------------------------------
UNAUTHORIZED ACCESS DETECTED

------------------------------------------------
Button 2 Pressed - Security system disarmed

------------------------------------------------
Button 3 Pressed - Sound sensor board disconnected from server
```

In [ ]:

In [ ]: